CMD命令

2025年8月8日 16:21

盘符切换：d： 盘符名称+冒号
查看当前路径下的内容： dir
进入单级目录：cd +空格+文件夹名称
回退到上一级目录：cd..（有两个点）
省打文件夹名：Tab
回退到盘符目录:cd \
清屏：cls
推出命令提示符窗口：exit

基础语法

2025年9月12日 19:46

1.注释

解释:对代码的简单说明

分类:a.单行注释--//注释信息 -- 即只能把注释写在一行内

b.多行注释--/*注释信息*/--即可以分成多行用/* */包括起来

c.文档注释--/**注释信息*/--

注意:不可嵌套

2.关键字

解释:被Java赋予了特定含义的英文单词

特点:a.关键字的字母全部小写

b.常用的代码编辑器,针对关键字有特殊的颜色标记,非常直观

主要的20个

class:用于(创建/定义)一个类--类是Java最基本的组成单元

意义:定义一个类,后面跟类名

3.字面量

解释:

字面量是 Java 中 “写死” 在代码里的固定值，编译器会根据其形式自动推断类型

字面量类型	说明	举例
整数类型	不带小数点的数字	666，-88
小数类型	带小数点的数字	13.14，-5.21
字符串类型	用双引号括起来的内容	“HelloWorld”，“黑马程序员”
字符类型	用单引号括起来的,内容只能有一个	‘A’，‘0’，‘我’
布尔类型	布尔值，表示真假	只有两个值：true，false
空类型	一个特殊的值，空值	值是：null

特殊字符:

\t (缩进 对齐 大空格) 制表符 在打印的时候,把前面字符串的长度补齐到8,或者8的整数倍

作用:使不同的打印内容打印出来时可以对齐 更加美观

使用:需要用引号包括,*System.out.println("name" + '\t'+"age")*

局限:不能完全对齐(或者上下行中少的字符串用空格代替) 得要用printf进行格式化要求 例

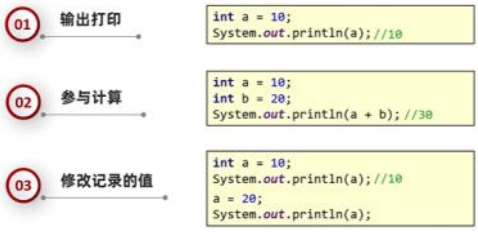
```
System.out.printf("%8s%8s\n", "name", "age");
```

变量:

格式:数据类型 变量名=数据值

(其中数据类型为空间中存储的数据,加入类型限制(整数或者小数) 例:整数用int 浮点数用 double)

使用方法:



注意:**在修改记录的值中 已经定义过一次变量后 如果想修改只要直接用变量名=数据值即可**

注意事项:1.只能存一个值

2.变量名不允许重复定义

3.一条语句可以定义多个变量,例:*int a=1,b=2,c=3;*

4.变量在使用之前一定要进行赋值

5.变量的作用域范围

数据类型

基本数据类型:

数据类型	关键字	取值范围
整数	byte	-128~127
	short	-32768~32767
	int默认	-2147483648~2147483647 (10位数)
	long	-9223372036854775808 ~ 9223372036854775807 (19位数)
浮点数	float	-3.401298e-38 到 3.402823e+38
	double默认	-4.9000000e-324 到 1.797693e+308
字符	char	0-65535
布尔	boolean	true，false

注意:如果要定义long类型的变量在数据后面需要加一共L作为后缀(建议L为大写)

定义float类型变量的时候数据值也要加一个F

标识符

定义:标识符是用来给变量、类、方法、接口、常量等程序元素命名的符号。它是程序中各个元素的唯一标识，以便在代码中引用它们。

硬性要求:1.由数字,字母,下划线_,和美元符号 $ 组成

2.不能以数字开头

3.不能是关键字

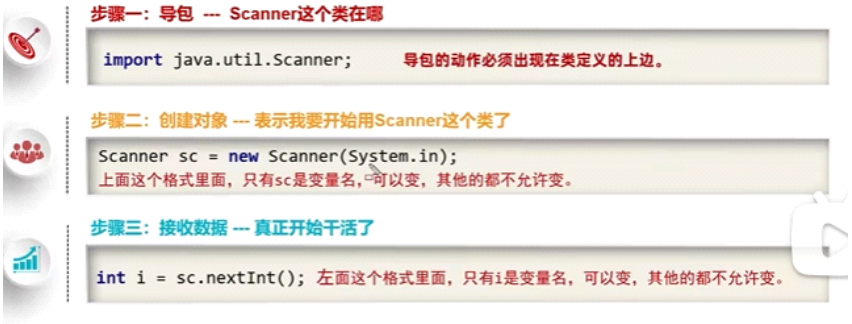
4.区分大小写

软性要求:

与用户交互--键盘录入

如何录入:导入一个类--Scanner,这个列就可以接受键盘输入的数字(类似与python的input函数)

步骤:



运算符

运算符定义:对字面量或者变量进行操作的符号

表达式定义:用运算符把字面量或者变量链接起来,符合Java语法的式子

例:

```
int a = 10;
int b = 20;
int c = a + b;
```

+ ：是**运算符**，并且是算术运算符

a + b：是**表达式**，由于+是算术运算符，所以这个表达式叫算术表达式

分类:1.算数运算符

2.自增自减运算符

3.赋值运算符

4.关系运算符

5.逻辑运算符

6.三元运算符

一.**<u>算数运算符</u>**:

注:在代码中,如果有小数参与计算,结果有可能是不准确的

	作用
+	加
-	减
*	乘

注:整数操作只能得到整数.要有小数 必须有浮点数参与运算

char类型参与运算使用的是计算机对应的十进制

+	加
-	减
*	乘
/	除
%	取模、取余

注:整数操作只能得到整数.要有小数 必须有浮点数参与运算

char类型参与运算使用的是计算机对应的十进制数值

byte, short和char类型将被提升到int类型,不管是否有其他类型参与运算.

从小范围到大范围的隐式转换等级顺序为:

Byte ,short, char,>int>long>float>double

对于+的操作:如果出现字符串 那么这个+就是字符的连接,而不是算术运算,并且是从左到右逐个执行

二.自增自减运算符

符号	作用	说明
++	自增	变量的值加1
--	自减	变量的值减1

++和--再单独使用时 放在变量前后结果相同

当参与多个操作时 ++和--放在变量后面就是先参与操作后加减

同理 当++和--放在变量前面就说先加减后参与操作

三.赋值运算符

符号	作用	说明
=	赋值	a=10，将10赋值给变量a
+=	加后赋值	a+=b，将a+b的值给a
-=	减后赋值	a-=b，将a-b的值给a
*=	乘后赋值	a*=b，将a×b的值给a
/=	除后赋值	a/=b，将a÷b的商给a
%=	取余后赋值	a%=b，将a÷b的余数给a

其中拓展的赋值运算符隐含了强制类型转换

四.关系运算符

符号	说明
==	a==b，判断a和b的值是否相等，成立为true，不成立为false
!=	a!=b，判断a和b的值是否不相等，成立为true，不成立为false
>	a>b，判断a是否大于b，成立为true，不成立为false
>=	a>=b，判断a是否大于等于b，成立为true，不成立为false
<	a<b，判断a是否小于b，成立为true，不成立为false
<=	a<=b，判断a是否小于等于b，成立为true，不成立为false

其中运算的结果都是布尔类型 不是true 就说false

五.逻辑运算符

符号	作用	说明
&	逻辑与	a&b，a和b都是true，结果为true，否则为false
|	逻辑或	a|b，a和b都是false，结果为false，否则为true
^	逻辑异或	a^b，a和b结果不同为true，相同为false
!	逻辑非	!a，结果和a的结果正好相反

其中运算的结果都是布尔类型

六.短路逻辑运算符

符号	作用	说明
&&	短路与	作用和&相同，但是有短路效果
||	短路或	作用和|相同，但是有短路效果

注:与逻辑运算符的区别是更高效

- 逻辑与&，无论左边真假，右边都要执行。
- 短路与&&，如果左边为真，右边执行；如果左边为假，右边不执行。
- 逻辑或|，无论左边真假，右边都要执行。
- 短路或||，如果左边为假，右边执行；如果左边为真，右边不执行。

七.三元运算符

格式: 关系表达式？表达式1：表达式2：

解释其中关系表达式的结果为布尔值,当结果为true时 调动表达式1 当结果为false时 调动表达式2

流程控制语句

一.总览:1.顺序结构

2.分支结构(if,switch)

3.循环结构(for,while,do...while)

二.顺序结构(最基础)

流程图:

三.分支结构

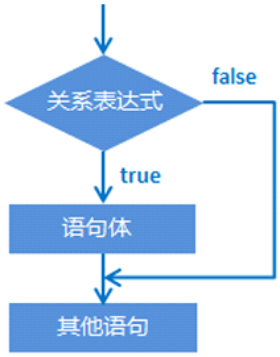
1. **i f**

(1) **if语句格式1**

格式:

```
if (关系表达式) {
语句体;
}
```

执行流程:



注:1.首先计算关系表达式的值

2.如果关系表达式的值为true就执行语句体

3.如果关系表达式的值为false 就跳过语句体

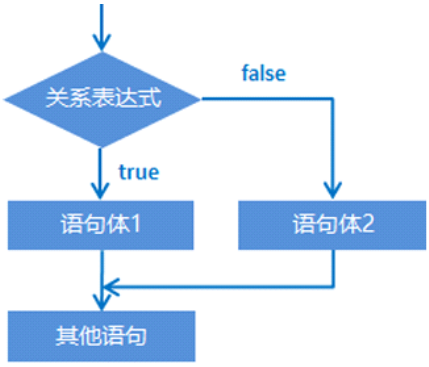
4.继续执行后面的语句内容

(2)**if 语句格式2**

格式:

```
if (关系表达式) {
        语句体1 ;
}       else {
        语句体2;
}
```

流程:



注:1.首先计算关系表达式中的值

2.如果关系表达式的值为true 就执行语句体1

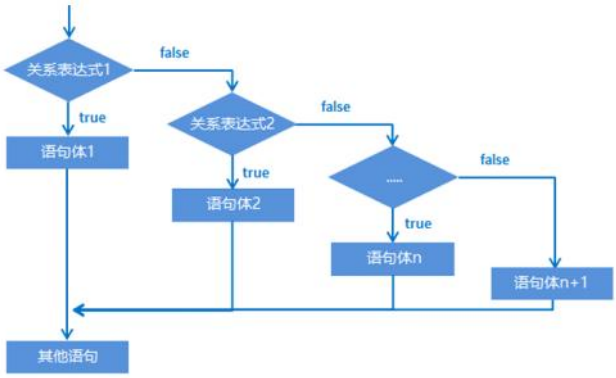
3.如果关系表达式的值为false 就执行语句体2

4.继续执行后面的语句内容

(3) if语句格式3

格式:

```
If (关系表达式1) {
        语句体1;
} else if (关系表达式2) {
        语句体2;
}
...
else{
        语句体n+1;
}
```



注: 1.首先计算表达式1的值

2.如果结果为true 就执行语句体1;如果值为false就计算关系表达式2的值

3.如果值为true就执行语句体2;如果值为false就执行关系表达式3的值

4....

5如果没有任何关系表达式为true 就执行语句体n+1

2. Switch

(1) switch语句1

格式:

```
Switch (表达式) {
    case 1;
        语句体1;
        break;
    case 2:
        语句体2;
        break;
    ...
    default:
        语句体n+1;
}
```

```java
public class SwitchDemo {
    public static void main(String[] args) {
        int week = 3;  // 假设今天是星期三（1=周一，2=周二..

        switch (week) {  // 这里的week是"判断变量"
            case 1:  // 如果week等于1（周一）
                System.out.println("今天上班");
                break;  // 执行完后跳出switch
            case 2:  // 如果week等于2（周二）
                System.out.println("今天上班");
                break;
            case 3:  // 如果week等于3（周三）
                System.out.println("今天开会");
                break;
            case 4:  // 如果week等于4（周四）
                System.out.println("今天上班");
                break;
            case 5:  // 如果week等于5（周五）
                System.out.println("今天上班");
                break;
            case 6:  // 如果week等于6（周六）
                System.out.println("今天休息");
                break;
            case 7:  // 如果week等于7（周日）
                System.out.println("今天休息");
                break;
            default:  // 如果上面的case都不匹配
                System.out.println("输入的星期不合法");
        }
    }
}
```

四.循环结构

1.for循环

for (初始化语句;条件判断语句;条件控制语句){

循环体语句;
}
注:**初始化语句**: 用于表示循环开始时的起始状态,简单来说就说循环开始的时候什么样什么样
条件判断语句:用于表示循环反复执行的条件,简单来说判断是否能一直执行下去
循环体语句: 用于表示循环反复执行的内容,简单说就是循环反复执行的事情
条件控制语句: 用于表示循环执行中每次变化的内容,简单说就是控制循环是否能一直执行下去

流程:1.执行初始化语句
2.执行条件判断语句,看其结果是true还是false
如果是false,循环结束
如果是true,循环执行
3.执行循环体语句
4.执行条件控制语句
5.回到2.继续
例:

```
public class ForLoopDemo {
    public static void main(String[] args) {
        // for循环的格式：for (初始化; 条件; 增量) { 循环体 }
        for (int i = 1; i <= 5; i++) {
            System.out.println("Hello Java " + i);  // 循环体：要重复做的事情
        }
    }
}
```

2.while循环
格式:
初始化语句;
While (条件判断句){
循环体语句;
条件控制语句;
}
流程:1.执行初始化语句
2.执行条件判断语句,看结果是true 还是false
如果是false,循环结束
如果是true,继续执行
3.执行循环体语句
4.执行条件控制语句
5.回到2继续

```
int score = 0;  // 初始分数0分（不及格）
// while循环格式：while (停止条件) { 重复做的事 }
while (score < 60) {  // 只要分数不及格，就继续刷题
    System.out.println("刷了1道题，分数增加10分");
    score = score + 10;  // 每次刷题后分数+10
}
System.out.println("终于及格了！分数是：" + score);
```

3.do...while循环
格式:

初始化语句;
do {
循环体语句;
条件控制语句;
}while(条件判断语句);
流程
1.执行初始化语句
2.执行循环体语句
3.执行条件控制语句
4.执行条件判断语句,看起结果是true还是false
如果是false 循环化结束
如果是true 继续执行
5.回到2继续

```java
import java.util.Random;
import java.util.Scanner;

public class DoWhileGuess {
    public static void main(String[] args) {
        Random random = new Random();
        int secretNumber = random.nextInt(10) + 1;  // 生成1-10的随机数（谜底）
        Scanner scanner = new Scanner(System.in);
        int guess;  // 你猜的数字

        // do...while: 先猜一次，再判断对不对
        do {
            System.out.print("请猜一个1-10的数字: ");
            guess = scanner.nextInt();  // 至少猜一次
        } while (guess != secretNumber);  // 如果没猜对，就再猜

        System.out.println("恭喜你，猜对了！谜底就是" + secretNumber);
    }
}
```

注:do...while与while的区别为 do...while是先干再说对不对

几种循环的区别

判断:

for和while的区别条件是是否知道停止条件,如果不知道就是for,若是知道就是while,

for计算得到的结果是结果量,while计算得到的结果是过程量

for 更擅长 “次数固定”，while 更擅长 “次数不固定但条件明确”。

使用:

条件控制与具有所控制的自增变量,因为归属for循环的语法结构中没在for循环结束后,就不能再次被访问

条件控制语句所控制的自增变量,队友while循环来说不归属其语法结构中,再while循环结束后,该变量还可以继续使用

4.死循环

```
for死循环格式 :
for(;;){

}

while死循环格式 :

while(true){

}

do..while死循环格式 :

do{

}while(true);
```

注:死循环是为了防止不明用户操作

死循环下面不能再写代码 因为永远执行不到

5.跳转控制语句

跳转控制语句1: break ----跳出循环,结束循环

跳转控制语句2:continue-----仅跳过本次循环,继续下次循环

注:continue 只能再循环中进行使用

数组

定义:是一种容器,可以用力啊存储同种数据类型的多个值

注:1.数组容器在存储数据的时候,需要结合隐式转换考虑

2.int类型的数组容器(byte short int)

3.double类型的数组容器(byte short int long float double)

建议容器的类型和存储的数据类型保持一致

数组的定义:

数组的静态初始化

初始化:就是在内存种,为数组容器开辟空间,并讲数据存入容器的过程

简化格式:数据类型[] 数组名 = {元素1,元素2,元素3...}

数组的地址值

在内存中数组的地址值表示数组在内存中的位置

例:[D@776ec8df]

其中 [ ] 表示这个是数组

D表示数据类型为double

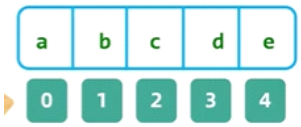
@单纯的无意义间隔符号 固定的格式

776ec8df 这才是真正的地址值 (16进制)

数组的元素访问

索引:也叫做下标

特点: 从0开始逐个增长+1



格式:数组名[索引]　例 arr[2]

把数据存储到数组中

格式 数组名[索引] = 具体数据/变量　例:arr[2] = 200

数组遍历

定义:把数组中所有内容去出来,取出来后可以(打印,求和,判断...)

通用格式: for(int x = 0 ; x<arr.length ; x++){
System.out.println(arr[x]);
}

补: 关于数组的长度属性用 length

使用格式 数组名.length

数组的动态初始化

格式:数据类型[ ] 数组名 = new 数据类型[数组的长度];

例: int [ ] member = new int [88]

数组动态初始化和静态初始化的区别

动态初始化 只明确元素个数 但是不确定具体值

静态初始化 已经完全明确要操作的具体数据

Java的内存分配

栈　方法运行时使用的内存,比如main方法运行,进入方法栈中执行

堆　存储对象或者数组,new来创建的,都存储在堆内存

方法区　存储可以运行的class文件

本地方法栈 jvm在使用操作系统功能的时候使用,和我们开发无关

寄存器　给cpu使用 和我们开发无关

方法

1.概念:是将具有独立功能的代码组织为一个整体,使其具有特殊功能的代码集

注意: 方法必须先建立才可以使用,该过程成为方法定义

方法创建后并不可直接运行,需要手动使用后才执行,此过程成为方法调用

2.1(无参数)定义:

格式: public static void 方法名 ( ) {}

2.2(含参数)定义:

格式:public static void 方法名 (参数){.. ..}

Public static void 方法名 (数据类型 变量名) {.. ..}

3.1(无参数)调用:

格式: 方法名 ( );

3.2(含参数)调用:

格式:方法名(参数);

方法名(变量名/常量值);

注:在定义中,参数是 数据类型+数据

形参和实参

1.形参:方法定义中的参数

等同于变量定义格式 例如 int number

2.实参:方法调用中的参数

等同于使用变量或常量

带返回值方法的定义和调用

定义格式:

Public static 数据类型 方法名(参数){

Return 数据;

}

常用API

一.Scanner类

next():用于读取输入的字符串 但是在输入的内容空格后的内容就无法记录了

nextLine():可以讲数据完整的接收过来

nextInt():用于读取输入的整数 但是如果遇到空格 将会结束当前的进程 但是会把空格的数保存下来再记录

二.String类

方法名	说明
public String()	创建一个空白字符串对象，不含有任何内容
public String(char[] chs)	根据字符数组的内容，来创建字符串对象
public String(String original)	根据传入的字符串内容，来创建字符串对象
String s = “abc”；	直接赋值的方式创建字符串对象，内容就是abc

额外补充1

2025年9月12日 19:46

不同进制在代码中的表现形式

二进制: 由0 1 组成 代码中以0b开头

十进制:由0-9组成 前面不加前缀

八进制: 由0-7组成 代码中以0开头

十六进制: 由0-9还有a-f组成 代码中以0x组成

进制的转化

1.任意进制转化到十进制:

公式：系数 * 基数的权次幂 相加

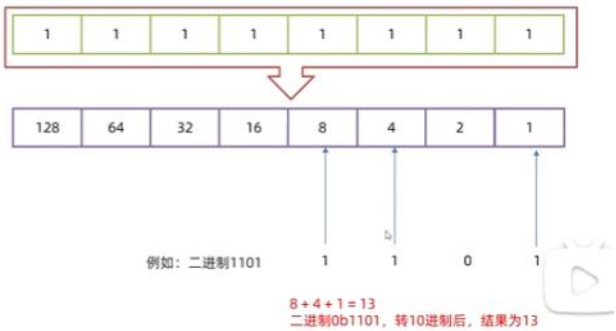
系数： 就是每一位上的数

基数： 当前进制数

权 ： 从右往左，依次为 0 1 2 3 4 5 ...

例:

8421快速转化法:



2.其他进制转到十进制:

除基取余法

不断的除以基数（几进制，基数就是几）得到余数，直到商为0，再将余数倒着拼起来即可。

代码逻辑理解

2025年9月12日 19:46

面向对象

1.方法的对象如果是一个类,那么调用此方法需要传入此类的对象或者此类的子类对象

2.多态:是对象多态和行为多态

3.多态成员访问特点: 成员方法 编译看左边(父类),运行看右边(子类)

[05-多态的优缺点_哔哩哔哩_bilibili]

方法的调用

1.在调用静态方法时 可以直接在main方法中直接用类名(参数);调用(因为main方法也是静态方法 静态方法只能访问静态成员)

如果在调用非静态方法时 需要先new当前类的对象,再用对象.方法名调用

2.